

```
!pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.2)
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (2.0.2)
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.16.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.5.2)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (0.1.6)
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.5.3)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (3.2.0)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

from imblearn.over_sampling import SMOTE

import tensorflow as tf
from tensorflow.keras import layers, regularizers
```

```
RANDOM_STATE = 42
```

```
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

```
print("Random seed fixed.")
print("TensorFlow version:", tf.__version__)
```

```
Random seed fixed.
TensorFlow version: 2.20.0
```

```
file_path = "/content/drive/MyDrive/ML_Project/Group03_Task2_DataSheet.xlsx"
```

```
df = pd.read_excel(file_path)
```

```
print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

```
df.head()
```

```
Dataset loaded successfully!
Shape: (70, 24)
```

	Sample No.	tanδ @ 0.000100009 Hz	tanδ @ 0.00021532 Hz	tanδ @ 0.00046404 Hz	tanδ @ 0.00100009 Hz	tanδ @ 0.0021543 Hz	tanδ @ 0.0046415 Hz	tanδ @ 0.01 Hz	tanδ @ 0.021544 Hz	tanδ @ 0.046416 Hz	...	tanδ @ 2.1546 Hz
0	1	0.313096	0.263748	0.216453	0.188339	0.194921	0.264037	0.428750	0.761844	1.390709	...	0.485241
1	2	0.308735	0.255590	0.200867	0.157484	0.132245	0.135262	0.165063	0.228676	0.358230	...	2.028017
2	3	0.307918	0.254060	0.197944	0.151697	0.120486	0.111080	0.115364	0.126730	0.148983	...	2.314646
3	4	0.571760	0.423162	0.330518	0.282384	0.273405	0.324580	0.477885	0.818407	1.469344	...	0.485744
4	5	0.565407	0.412576	0.311466	0.246326	0.203263	0.185103	0.197077	0.255297	0.391262	...	2.028166

```
5 rows × 24 columns
```

```
# Column 1 = Sample number
sample_no = df.iloc[:, 0]
```

```
# Columns 2 to 23 = 22 FDS tan(delta) features
X = df.iloc[:, 1:23].values.astype(float)
```

```
# Class column
y = df["Class"].astype(str).str.strip().values
```

```
classes = ["Dry", "Intermediate", "Wet"]

print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nOriginal class distribution:")
print(pd.Series(y).value_counts())
```

```
X shape: (70, 22)
y shape: (70,)
```

```
Original class distribution:
Wet      24
Dry      23
Intermediate  23
Name: count, dtype: int64
```

```
X_train_real, X_test_real, y_train_real, y_test_real, sample_train, sample_test = train_test_split(
    X,
    y,
    sample_no,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y
)
```

```
print("Real training samples:", X_train_real.shape[0])
print("Real testing samples:", X_test_real.shape[0])
```

```
print("\nTraining class distribution:")
print(pd.Series(y_train_real).value_counts())
```

```
print("\nTesting class distribution:")
print(pd.Series(y_test_real).value_counts())
```

```
Real training samples: 56
Real testing samples: 14
```

```
Training class distribution:
Intermediate  19
Wet          19
Dry          18
Name: count, dtype: int64
```

```
Testing class distribution:
Dry      5
Wet      5
Intermediate  4
Name: count, dtype: int64
```

```
X_train_log = np.log10(X_train_real)
X_test_log = np.log10(X_test_real)
```

```
print("Log transformation completed.")
```

```
Log transformation completed.
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train_log)
X_test_scaled = scaler.transform(X_test_log)
```

```
print("Standardization completed.")
print("Training shape:", X_train_scaled.shape)
print("Testing shape:", X_test_scaled.shape)
```

```
Standardization completed.
Training shape: (56, 22)
Testing shape: (14, 22)
```

```
label_encoder = LabelEncoder()
```

```
y_train_encoded = label_encoder.fit_transform(y_train_real)
y_test_encoded = label_encoder.transform(y_test_real)
```

```
print("Class mapping:")
for i, cls in enumerate(label_encoder.classes_):
    print(i, "->", cls)
```

```
Class mapping:
0 -> Dry
1 -> Intermediate
2 -> Wet
```

```

target_per_class = {
    label_encoder.transform(["Dry"])[0]: 93,
    label_encoder.transform(["Intermediate"])[0]: 93,
    label_encoder.transform(["Wet"])[0]: 94
}

smote = SMOTE(
    sampling_strategy=target_per_class,
    random_state=RANDOM_STATE,
    k_neighbors=5
)

X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train_encoded)

print("SMOTE completed.")
print("SMOTE training shape:", X_train_smote.shape)

unique, counts = np.unique(y_train_smote, return_counts=True)

print("\nSMOTE class distribution:")
for label, count in zip(unique, counts):
    print(label_encoder.inverse_transform([label])[0], ":", count)

print("\nTotal SMOTE training samples:", len(y_train_smote))

```

```

SMOTE completed.
SMOTE training shape: (280, 22)

SMOTE class distribution:
Dry : 93
Intermediate : 93
Wet : 94

Total SMOTE training samples: 280

```

```

X_nn_train, X_nn_val, y_nn_train, y_nn_val = train_test_split(
    X_train_smote,
    y_train_smote,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_train_smote
)

print("NN training samples:", X_nn_train.shape[0])
print("NN validation samples:", X_nn_val.shape[0])
print("Final real test samples:", X_test_scaled.shape[0])

```

```

NN training samples: 224
NN validation samples: 56
Final real test samples: 14

```

```

tf.random.set_seed(RANDOM_STATE)

nn_smote_model = tf.keras.Sequential([
    layers.Input(shape=(22,)),

    layers.Dense(
        32,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001)
    ),
    layers.Dropout(0.25),

    layers.Dense(
        16,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001)
    ),
    layers.Dropout(0.20),

    layers.Dense(
        8,
        activation="relu",
        kernel_regularizer=regularizers.l2(0.001)
    ),

    layers.Dense(3, activation="softmax")
])

nn_smote_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),

```

```
        loss="sparse_categorical_crossentropy",  
        metrics=["accuracy"]  
    )  
  
nn_smote_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	736
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 8)	136
dense_3 (Dense)	(None, 3)	27

Total params: 1,427 (5.57 KB)
Trainable params: 1,427 (5.57 KB)
Non-trainable params: 0 (0.00 B)

```
early_stop = tf.keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    patience=30,  
    restore_best_weights=True  
)  
  
history = nn_smote_model.fit(  
    X_nn_train,  
    y_nn_train,  
    validation_data=(X_nn_val, y_nn_val),  
    epochs=300,  
    batch_size=16,  
    callbacks=[early_stop],  
    verbose=1  
)
```

```

14/14 0s 17ms/step - accuracy: 0.9888 - loss: 0.0673 - val_accuracy: 1.0000 - val_loss: 0.0548
Epoch 296/300
14/14 0s 18ms/step - accuracy: 0.9911 - loss: 0.0752 - val_accuracy: 1.0000 - val_loss: 0.0504
Epoch 297/300
14/14 0s 20ms/step - accuracy: 0.9955 - loss: 0.0670 - val_accuracy: 1.0000 - val_loss: 0.0497
Epoch 298/300
14/14 0s 23ms/step - accuracy: 0.9955 - loss: 0.0729 - val_accuracy: 1.0000 - val_loss: 0.0585
Epoch 299/300
14/14 0s 8ms/step - accuracy: 1.0000 - loss: 0.0585 - val_accuracy: 1.0000 - val_loss: 0.0492
Epoch 300/300
14/14 0s 8ms/step - accuracy: 1.0000 - loss: 0.0584 - val_accuracy: 1.0000 - val_loss: 0.0483

```

```

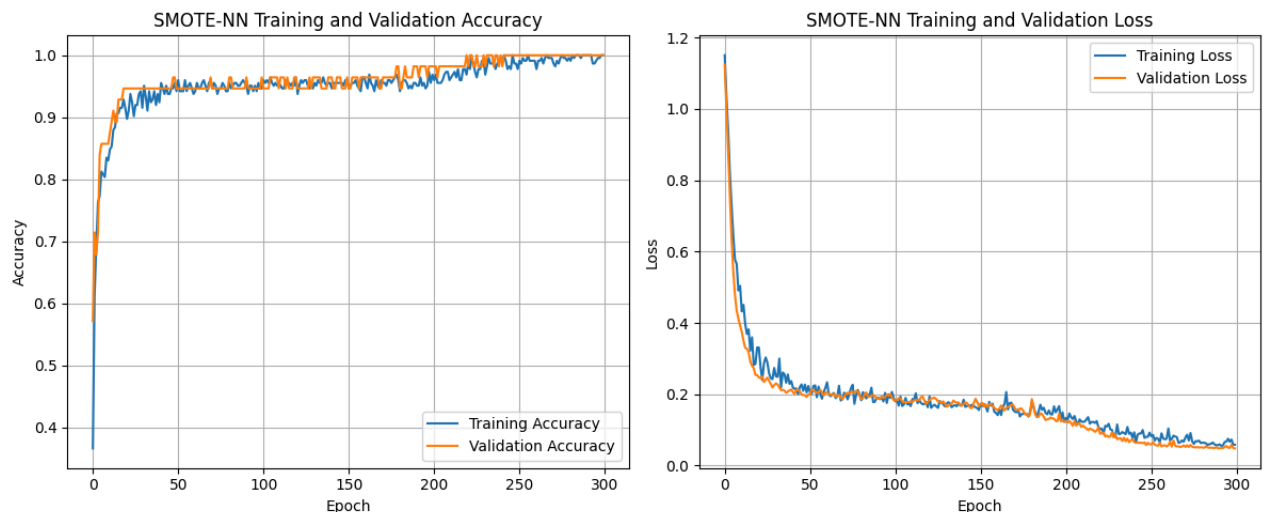
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("SMOTE-NN Training and Validation Accuracy")
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("SMOTE-NN Training and Validation Loss")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

```



```

train_loss, train_acc = nn_smote_model.evaluate(X_nn_train, y_nn_train, verbose=0)
val_loss, val_acc = nn_smote_model.evaluate(X_nn_val, y_nn_val, verbose=0)
test_loss, test_acc = nn_smote_model.evaluate(X_test_scaled, y_test_encoded, verbose=0)

print(f"SMOTE-NN Training Accuracy : {train_acc * 100:.2f}%")
print(f"SMOTE-NN Validation Accuracy : {val_acc * 100:.2f}%")
print(f"SMOTE-NN Real Testing Accuracy: {test_acc * 100:.2f}%")

print(f"\nSMOTE-NN Training Loss : {train_loss:.4f}")
print(f"SMOTE-NN Validation Loss : {val_loss:.4f}")
print(f"SMOTE-NN Real Testing Loss: {test_loss:.4f}")

```

```

SMOTE-NN Training Accuracy : 100.00%
SMOTE-NN Validation Accuracy : 100.00%
SMOTE-NN Real Testing Accuracy: 92.86%

```

```

SMOTE-NN Training Loss : 0.0493
SMOTE-NN Validation Loss : 0.0483
SMOTE-NN Real Testing Loss: 0.6022

```

```
y_test_prob = nn_smote_model.predict(X_test_scaled)
```

```

y_test_pred_encoded = np.argmax(y_test_prob, axis=1)

y_test_pred = label_encoder.inverse_transform(y_test_pred_encoded)

prediction_df = pd.DataFrame({
    "Sample_No": sample_test.values,
    "Actual_Class": y_test_real,
    "Predicted_Class": y_test_pred
})

prediction_df

```

1/1 ————— 0s 187ms/step

	Sample_No	Actual_Class	Predicted_Class	
0	46	Intermediate	Wet	
1	15	Dry	Dry	
2	45	Intermediate	Intermediate	
3	35	Intermediate	Intermediate	
4	48	Wet	Wet	
5	40	Intermediate	Intermediate	
6	8	Dry	Dry	
7	55	Wet	Wet	
8	60	Wet	Wet	
9	64	Wet	Wet	
10	20	Dry	Dry	
11	50	Wet	Wet	
12	7	Dry	Dry	
13	11	Dry	Dry	

Next steps: [Generate code with prediction_df](#) [New interactive sheet](#)

```

print("SMOTE-NN Classification Report on Real Test Data:")
print(classification_report(y_test_real, y_test_pred, labels=classes))

```

```

SMOTE-NN Classification Report on Real Test Data:
              precision    recall  f1-score   support

   Dry           1.00        1.00        1.00         5
 Intermediate    1.00        0.75        0.86         4
    Wet          0.83        1.00        0.91         5

 accuracy              0.93         14
  macro avg           0.94        0.92        0.92         14
 weighted avg           0.94        0.93        0.93         14

```

```

cm_smote_nn = confusion_matrix(y_test_real, y_test_pred, labels=classes)

plt.figure(figsize=(6, 5))
plt.imshow(cm_smote_nn, cmap="Blues")

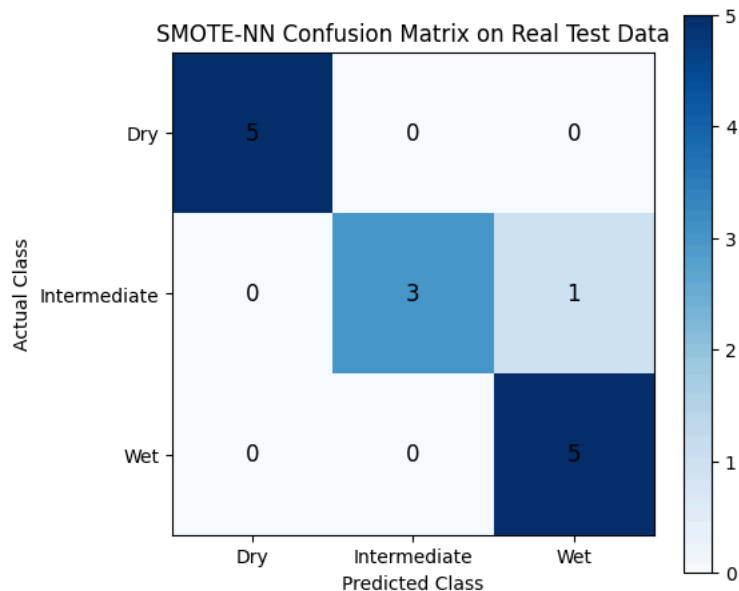
plt.title("SMOTE-NN Confusion Matrix on Real Test Data")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(np.arange(len(classes)), classes)
plt.yticks(np.arange(len(classes)), classes)

for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm_smote_nn[i, j], ha="center", va="center", fontsize=12)

plt.colorbar()
plt.tight_layout()
plt.show()

```



```
output_folder = "/content/drive/MyDrive/ML_Project/smote_nn_outputs"
os.makedirs(output_folder, exist_ok=True)

prediction_df.to_excel(f"{output_folder}/smote_nn_test_predictions.xlsx", index=False)

with open(f"{output_folder}/smote_nn_classification_report.txt", "w") as f:
    f.write("SMOTE-NN Classification Report\n")
    f.write("-----\n\n")
    f.write(f"SMOTE training samples: {len(y_train_smote)}\n")
    f.write(f"Original real test samples: {len(y_test_real)}\n")
    f.write(f"Training Accuracy : {train_acc * 100:.2f}%\n")
    f.write(f"Validation Accuracy : {val_acc * 100:.2f}%\n")
    f.write(f"Real Testing Accuracy: {test_acc * 100:.2f}%\n\n")
    f.write(classification_report(y_test_real, y_test_pred, labels=classes))

print("Results saved to:")
print(output_folder)
```

Results saved to:
/content/drive/MyDrive/ML_Project/smote_nn_outputs